

SAI Chatbot — Roadmap to Real-Time Data

To: CEO **From:** Engineering **Subject:** Static → Real-Time: where the bot is today, and the work to make it live **Status:** Phase 1 (Static) operational · Phase 2 (New Data Structure) in progress · Phase 3 (Webhook) planned

1. Executive Summary

The **SAI (Smart Assistant for IoT)** chatbot is **live and working today** using a static snapshot of the bank's IoT data. The bot can already answer ~80% of operational questions — branch health, alerts, gateway status, CCTV, power — by reading from a pre-built snapshot instead of receiving live events.

We are now in the middle of **Phase 2: re-platforming the data layer** to a normalized per-branch model. Once that lands, **Phase 3: webhook integration** will make every ThingsBoard event flow into the bot within seconds, in real time.

What this means for the business:

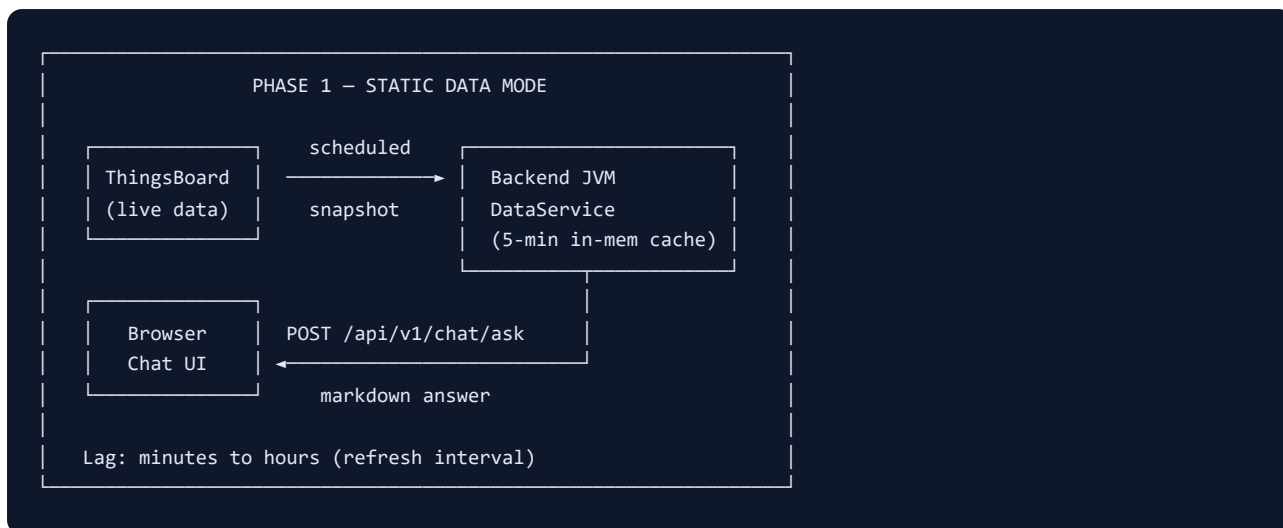
- **Today (Phase 1):** Answers are correct, but lag the ThingsBoard dashboard by the refresh cycle of the snapshot (minutes to hours, depending on deployment).
- **Within ~2 sprints (Phase 2 done):** The data structure is clean and indexed; bot answers stay correct as we wire up live data.
- **Within ~3–4 sprints from today (Phase 3 done):** Bot answers match the dashboard in **near real-time** with no manual refresh.

No customer-facing downtime. The same chat surface, the same multi-tenant dashboard embedding, the same 10-bank rollout.

2. Current State — Phase 1 (Static Data)

Status: Working in production.

The bot reads from a snapshot of all customer devices (`GET /api/v1/data/full`) that the backend refreshes on a schedule. The React chat widget inside the ThingsBoard dashboard posts a question to `POST /api/v1/chat/ask`, the backend classifies the intent, and returns a deterministic markdown answer.



What works:

- Multi-tenant chat (10 banks, hard-coded prefixes: `SYNB` , `UBIN` , `IOB` , `CBI` , ...)
- Deterministic answer engine with 25+ intents
- OpenAI `gpt-4o-mini` fallback for free-form questions
- Voice input, glassmorphism UI, iframe-embedded in ThingsBoard
- Per-user branch index with alias matching
- Nightly reconciliation that rebuilds the snapshot from history

What is the limitation:

- Answers are only as fresh as the last snapshot pull.
- New events appear in the bot only after the next refresh tick.

3. Phase 2 — New Data Structure (In Progress)

Status: Engineered, partially landed, being stabilized.

We are replacing the flat `branchName.field` payload with a typed, normalized per-branch domain model. This is the prerequisite for real-time because:

- The deterministic engine needs to know **what kind of value** it is reading (state vs. number vs. boolean).
- Field names differ across tenants; we need a single canonical shape.
- Status transitions (ON → OFF, ONLINE → OFFLINE) need to update counters atomically across the whole branch and the global view.

PHASE 2 — NORMALIZED DATA MODEL

Raw payload (messy)	Normalized BranchSnapshot (clean)
"AHD_Vastrapur.gateway Status_battery = 12.4"	BranchIdentity{ customerCode="SYNB" branchName="AHD_Vastrapur" parentPath="Gujarat/Ahmedabad" }
"AHD_Vastrapur.power_ status = ON"	GatewayStatus{ battery=12.4 }
"AHD_Vastrapur.camera_ online = 1"	PowerStatus{ mains=ON }
...	CctvStatus{ online=true }
	AlertSummary{ active=2 }

Pipeline: FullDataPayloadParser → FieldPrecedenceResolver
→ ValueNormalizer → BranchSnapshotMapper

Plus: BranchAliasIndex for fuzzy "vastrapur" ↔ "AHD_Vastrapur"

Components completed:

- Domain model (BranchSnapshot , GatewayStatus , PowerStatus , CctvStatus , BranchSubsystems , AlertSummary , HardwareHealth)
- Normalization pipeline (FullDataPayloadParser , ValueNormalizer , FieldPrecedenceResolver , BranchSnapshotMapper)
- Branch alias index
- Golden-question accuracy test suite

What this enables:

- The query engine reads typed values; no more "is this a string or a number?" guessing.
- Status transitions become first-class: ON→OFF updates a counter, not just a field.
- Per-tenant data differences are absorbed by the precedence resolver instead of breaking the answer.

4. Phase 3 — Webhook Implementation (Planned)

Status: Code path exists, not yet wired to live TB.

Once Phase 2 is stable, we turn on the real-time pipeline. ThingsBoard rules will fire on every relevant event and POST to our `WebhookController` . The consumer JVM picks the event up from RabbitMQ, deduplicates it, persists it to TimescaleDB, and atomically updates the Redis state that the chat path reads.



Component-level flow:



```

└─ ReconciliationService (nightly @Scheduled cron)
    └─ ReplayService (admin trigger) — replays device_events

```



Key properties:

- **Idempotent.** Each event UUID is checked against Redis (`SETNX` with 24h TTL) before processing — duplicate deliveries are dropped.
- **Atomic counters.** A Lua script (`update_counters.lua`) decrements the old status bucket and increments the new one in a single Redis round-trip — no race conditions across concurrent events.
- **Replayable.** The full event log in TimescaleDB lets us rebuild Redis from scratch via `ReplayService` if needed.
- **Self-healing.** A nightly `@Scheduled` reconciliation in `ReconciliationService` (cron `0 0 2 * * ? Asia/Kolkata`) re-derives Redis from the last 24h of events and logs any drift.

What changes for the user:

- A power cut at a branch shows up in the bot within seconds, not minutes.
- An alert raised in ThingsBoard is reflected in the next question the user asks.
- No frontend changes — same chat widget, same UI.

5. Work Breakdown & Effort Estimate

This is a realistic look at the engineering work between today and the real-time launch.

Already done (Phase 1)

Area	Work
Backend	Spring Boot 4.0.3 service, multi-tenant, REST + AMQP
Chat engine	25+ intents, deterministic answer engine, OpenAI fallback
Frontend	React 18 + Tailwind v4, voice input, iframe-embedded
Snapshot pipeline	<code>DataService</code> with 5-min cache
Tests	Golden-question accuracy test, normalization tests, controller tests
Infra	Dockerfile, docker-compose, Caddy reverse proxy

Phase 2 (now → 2 sprints)

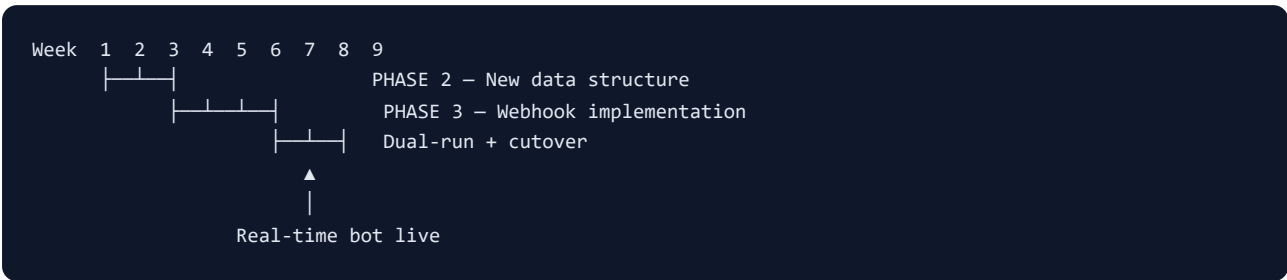
Task	Estimate	Notes
Stabilize <code>BranchSnapshotMapper</code> against all tenants	1 days	Per-tenant data quirks
Tighten <code>FieldPrecedenceResolver</code> rules	1 days	
Promote <code>BranchAliasIndex</code> from soft to authoritative	2 days	
Increase golden-question accuracy to > 95%	2 days	Iterative
Performance test on 24h synthetic payload	1 days	
Subtotal	~7 working days	

Phase 3 (2 sprints after Phase 2)

Task	Estimate	Notes
HMAC signature verification on <code>WebhookController</code>	1 day	Security: critical, not optional
Configure TB-side webhook rules (10 tenants)	2 days	One per bank; reusable template
Per-customer queue + binding declarations in <code>RabbitMQConfig</code>	1 day	Idempotent; safe to redeploy
Lua script hardening + Redis cluster failover test	2 days	
ReconciliationService tuning + on-call runbook	2 days	
End-to-end chaos test: kill consumer, replay events	2 days	
Cutover plan: dual-run for 1 week, then flip	2 days	Zero-downtime switch
Subtotal	~12 working days	

Total remaining effort: ~25 working days (≈ 5 calendar weeks, 1 engineer)

6. Timeline



- **Today:** Bot in production on static data.
- **End of Week 1:** New data structure stable, golden-questions > 95%.
- **End of Week 1:** Webhook pipeline code-complete.
- **End of Week 2:** Dual-run for one customer (pilot bank), then full cutover.
- **End of Week 2:** Real-time bot fully live across all 10 tenants.

7. Business Impact

Dimension	Today (Static)	After Phase 3 (Real-Time)
Time to reflect a new alert in chat	Minutes – hours (snapshot interval)	Seconds
Ops use case	"What's the status right now?" answered with last-known	Live, same as dashboard
Customer trust	"It lags a bit"	"It knows"
OpenAI spend	Low (deterministic-first)	Low (no change — same engine)
Infra cost	Single JVM	Chat JVM + Ingestion JVM + Consumer JVM (horizontal)
Downtime risk	Low	Low (dual-run cutover)

The deterministic engine is the same in both phases; we are not rewriting the brain, only the data feed. That keeps risk contained and lets us ship the real-time cutover with confidence.

8. Risks & Mitigations

Risk	Mitigation
Webhook spoofing / injection	HMAC signature verification on every request (Phase 3, day 1)
Duplicate events from RabbitMQ	IdempotencyService (Redis <code>SETNX</code> , 24h TTL) — already built
Redis flush = data loss	TimescaleDB event log + <code>ReplayService</code> rebuild
Per-tenant data quirks breaking the answer	<code>FieldPrecedenceResolver</code> + <code>BranchAliasIndex</code> + golden-question regression
JVM time-zone drift on the nightly cron	Hard-coded <code>zone="Asia/Kolkata"</code> on the <code>@Scheduled</code> annotation
Cutover surprises	One-week dual-run with a pilot bank before flipping all 10

9. What I'm Asking For

- **Approval to proceed** with Phase 3 as scoped (~12 engineering days).
- **One pilot bank** for the dual-run cutover (recommend **SYNB** — largest tenant, best telemetry coverage).
- **Continued access** to a ThingsBoard sandbox tenant for chaos-testing the webhook path.

No additional headcount required. No new vendors. The same React + Spring stack we already run.
